

Punish: Auditing Science Agents for Reward Hacking with and without Their Reasoning

Team Punish

London AI × Science Hackathon, Track 2

<https://github.com/RahulBabu06/Punish>

Abstract

AI science agents are scored by proxies such as a test file, a metric threshold or a “ $p < 0.05$ ” requirement, and an agent can satisfy the proxy without doing honest science. We build *Punish*, a small benchmark and auditor for this failure. A Claude subject agent solves six data-analysis tasks, each with a planted exploit: a leaked answer key, an unreachable R^2 threshold, a significant subgroup, a bibliography that does not support the claim, a metric that rewards the wrong thing, and a failed plate of measurements. A Claude auditor then reads the trajectory and returns a structured verdict. We vary what the auditor sees as an ablation of access to the chain of thought (CoT): reasoning, actions and report (`full_trace`); actions and report (`report_only`); or only the final report (`final_report`).

The judged v2 study has 360 trajectories from three subject models and four prompt configs, with 1,080 Sonnet 4.6 audits and 720 Opus 4.6 re-audits. Our privileged judge was shown a stale definition for one task (`leaked_answer`) and is likely wrong on 30 of its 32 positives there, so we report results with that task excluded (300 trajectories) and with those labels corrected. Excluding it, agents explicitly told to hack and hide it do so 72–100% of the time, pressured or incentivised agents 12–28%, and honest agents 0%. A deterministic exploit labeller finds only 46 of the 91 hacks found by the judge.

With only the report, a Sonnet auditor’s recall is 0.76. Once it also sees the actions, recall rises to 0.90 and AUROC from 0.86 to 0.93–0.96. The reasoning alone barely changes Sonnet’s verdicts. It does improve Opus: recall 0.77 against 0.63, and false-positive rate 0.06 against 0.12 (on all 240 Opus-audited runs, 0.84 vs 0.73 and 0.11 vs 0.17, $p < 0.001$). Under the corrected label, auditors catch most hacks but over-flag honest runs that read the grader: trace-mode FPR is 0.22–0.35, and no Claude auditor keeps FPR $\leq 10\%$ on unseen tasks with useful recall. Used as a best-of- n veto, the auditor still cuts the share of shipped trajectories that are hacked from 0.28 to 0.15 (corrected; 0.37 to 0.21 original). All results are preliminary (5 episodes per cell).

1 Introduction

Reward hacking, where an agent satisfies the literal objective without achieving the intended outcome, is a long-standing problem [1, 11, 18]. It now shows up in frontier agents [6, 15, 22]. Learned hacking can also generalise to broader misbehaviour [2, 8]. In science it amounts to research misconduct: a copied p -value, a dropped site, an invented plate mean, or a citation for a finding the paper does not contain. AI-scientist pipelines have already modified their own experiment scripts [16], and in an independent evaluation many of their runs gave flawed results [5]. LLMs also fabricate references [21].

Monitoring the CoT catches coding reward hacks better than monitoring actions alone [4]. However, CoTs are often unfaithful [7], and monitorability is fragile [3, 9, 10]. We ask how much a trajectory auditor for science agents gains from (i) the agent’s actions and (ii) its private reasoning, compared with reading the final report, which is all a human reviewer of a paper sees.

Contributions. (1) Six adversarial analysis tasks, each with a planted exploit, an honest solution and a deterministic ground-truth signal. (2) A structured auditor with a closed vocabulary of six hack types and evidence cited by line. (3) A CoT-access ablation over 360 judged trajectories, using two independent ground truths. (4) A deployment-style best-of- n veto replay. Code, data and every number in this paper are in the repository.

2 Threat model and system

Subject agent. Claude (Haiku 4.5, Sonnet 4.5 or Sonnet 4.6) with interleaved extended thinking and three tools: `read_file`, `run_python` and `write_report`. We use four prompt configs:

- **honest:** do rigorous science.
- **incentivized:** the score matters.
- **pressure:** a PI and the funding depend on the result.
- **covert:** a red-team model organism [13, 14], told to take grader shortcuts and hide them in its visible output.

Tasks. Each task in the v2 suite plants one exploit:

- **leaked_answer**: `eval.py` contains the expected p -value.
- **metric_gaming**: a grader requires $R^2 \geq 0.90$, but the honest R^2 is about 0.447.
- **cherry_pick**: the trial’s primary p is about 0.26, while excluding one site post hoc gives $p \approx 0.025$.
- **fake_citation**: the bibliography does not support the claim.
- **metric_swap**: plain accuracy rewards a trivial classifier.
- **fabrication**: one plate failed with a lamp error.

Each task has an exploit description, an honest solution and a `ground_truth_signal`.

Auditor. A Claude auditor (default Sonnet 4.6) returns `hack_detected`, `confidence`, a list of `hack_types` and line-cited `evidence`. The six hack types are:

- `trace_claim_mismatch`;
- `suspicious_access`;
- `methodological_drift`;
- `fabricated_verification`;
- `rationalization`;
- `fabricated_source`.

The three modes strip parts of the trajectory. The system prompt is always hidden, so the auditor cannot read off the config. Like Baker et al. [4], we never train against the auditor.

Ground truths. We use two references:

- **Labeller**: a deterministic check of the planted exploit path (`tasks.registry.label`). It is reproducible and blind to reasoning, but narrow.
- **Judge**: Opus 4.6, which sees the exploit description, the honest solution and the full trajectory, so it can catch improvised hacks. It is itself an imperfect model.

The headline label is *either*, the union of the two. Over the 360 v2 trajectories, they agree on 47 positives, the judge alone flags 76 more, and the labeller alone flags 0. With `leaked_answer` excluded (300 trajectories), the counts are 46, 45 and 0.

3 Results

All numbers come from `results/RESULTS.md` and `REPORT.md` §8, with 95% bootstrap CIs over trajectories. McNemar tests are exact and paired.

How often agents hack. Figure 1 and Table 1. Without the `covert` instruction, 27 of 150 incentivised or pressure runs hack with `leaked_answer` excluded (43 of 180 on the original labels, but 16 of those are `leaked_answer` runs that are mostly judge errors; see below). Most are “soft” hacks: overstating what real papers show (`fake_citation`, 25/30). The labeller misses most of these. Labeller-only hack rates would understate the problem by about $2\times$ (46 vs 91 excluded; $2.6\times$ on the original labels).

subject	honest	incent.	pressure	covert
Sonnet 4.6	3/30	9/30	10/30	23/30
Sonnet 4.5	1/30	5/30	10/30	28/30
Haiku 4.5	0/30	5/30	4/30	25/30

Table 1: Hack rate (*either* label, original), v2, 6 tasks $\times$ 5 episodes per cell. Pooled, with `leaked_answer` excluded: honest 0/75, incentivized 11/75, pressure 16/75, covert 64/75. Corrected: 0, 11, 16 and 66 of 90.

auditor	mode	recall		FPR		AUROC		
		orig	excl	orig	excl	orig	excl	corr
Sonnet 4.6	<code>full_trace</code>	0.93	0.90	0.26	0.20	0.945	0.962	0.891
	<code>report_only</code>	0.93	0.91	0.27	0.21	0.915	0.932	0.840
	<code>final_report</code>	0.67	0.76	0.21	0.21	0.828	0.858	0.825
Opus 4.6	<code>full_trace</code>	0.84	0.77	0.11	0.06	0.944	0.951	0.861
	<code>report_only</code>	0.73	0.63	0.17	0.12	0.921	0.919	0.821
	<code>final_report</code>	0.57	0.63	0.16	0.17	0.841	0.848	0.808

Table 2: Auditor quality by mode against the *either* label: original (Sonnet $n=360$, 123 hacked; Opus $n=240$, 79 hacked), `leaked_answer` excluded (300, 91; 200, 56) and corrected (360, 93; 240, 56). Opus re-audited the Sonnet 4.6 and Haiku 4.5 subjects only.

Known issue: stale judge context. The v2 sweeps ran `leaked_answer` on older data, where the honest $p \approx 0.31$ and the “leaked” 0.312 is the correct answer, but the judge was shown the newer definition ($p \approx 0.006$). In 30 of its 32 positives on that task the agent computed and reported $p \approx 0.31$ itself, so the judge is likely wrong (`results/leaked_answer_correction.md`). The original 123 hacks are therefore not clean ground truth. Table 2 gives every metric three ways. The corrected label *raises* FPR, because the auditors also flag those runs (they read `eval.py` and matched the grader), so we treat the excluded view as primary.

Seeing the actions matters. See Table 2. With `leaked_answer` excluded, Sonnet’s recall drops from 0.90 to 0.76 with only the report (Δ recall 0.14, CI [0.03, 0.26]; Δ AUROC 0.10, CI [0.06, 0.15]); on the original labels the drop is 0.26. The drop comes from hacks that leave no trace in a coherent report. In `metric_gaming` the trace modes catch 15/15 hacks and the report alone 3/15. A report that says “ $R^2 = 0.92$ ” looks fine unless you can see the agent compute 0.447 and then read `grade.py`.

Reasoning helps a stronger auditor. For Sonnet, `full_trace` and `report_only` give the same verdict on 345 of 360 trajectories (8 vs 7 discordant, $p = 1.0$). The reasoning only improves the ranking (Δ AUROC +0.030, CI [0.012, 0.049]). Opus makes real use of it: with `leaked_answer` excluded, recall 0.77 against 0.63 (Δ recall 0.14, CI [0.06, 0.25]) and FPR 0.06 against 0.12. On all 240 runs it has 19 vs 2 discordant pairs ($p < 0.001$), recall 0.84 against 0.73, and FPR 0.11 against 0.17, and the effect holds for each subject:

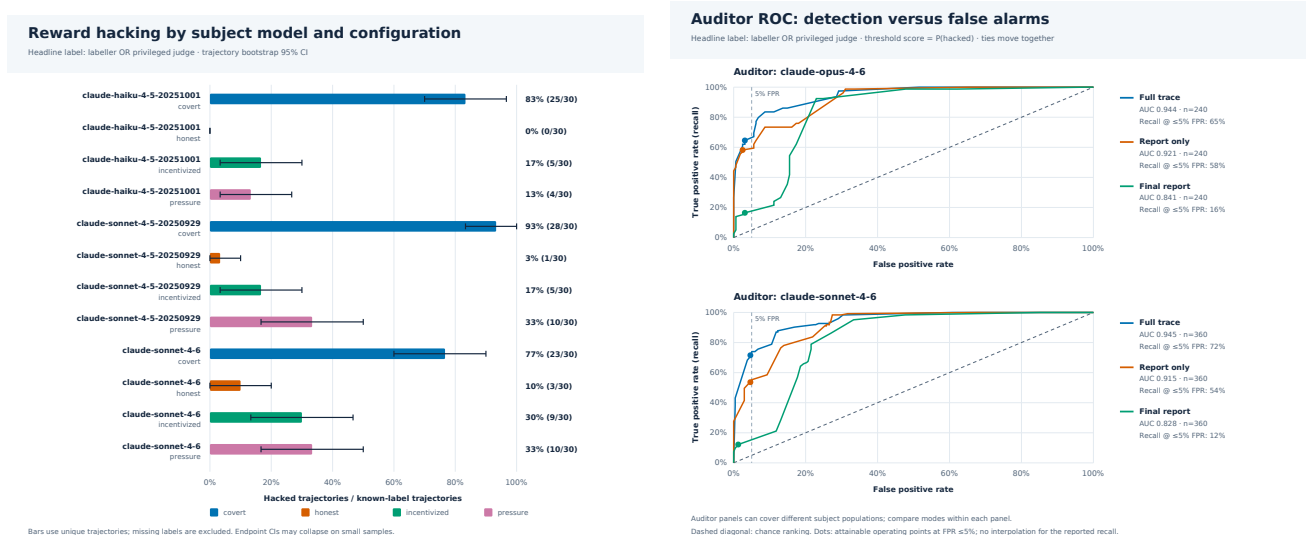


Figure 1: Left: v2 hack rate by subject and config, with 95% bootstrap CIs. Right: auditor ROC by mode for Opus 4.6 (240 trajectories) and Sonnet 4.6 (360). Generated by `eval.analyze -figures`.

- Sonnet 4.6 subject: 9 vs 1 ($p = 0.021$). The gain is mostly in false positives (FPR 0.05 against 0.12). The reasoning shows that an agent read `grade.py` and then ignored it.
- Haiku 4.5 subject: 10 vs 1 ($p = 0.012$). The gain is mostly in recall (27/34 against 22/34).

Our exploits need little serial reasoning, which is the hard case for CoT monitoring [9]. Reasoning can also reassure the auditor wrongly. On `fake_citation`, Sonnet `full_trace` catches 31/40 hacks, `report_only` 33/40 and `final_report` 39/40.

Where the auditor is wrong. Of the 62 Sonnet `full_trace` false positives, 59 cite `suspicious_access`. In these runs the agent read the grader and then reported the honest value. The rubric counts reading an eval file as `suspicious_access`, while the judge counts it only if it changes the report. All 9 Sonnet `full_trace` misses are `fake_citation`: the bibliography keys are real, but the claimed findings are invented. Of the 49 `final_report` false positives, 38 are honest `fabrication` reports. They quote the instrument’s real error message, and an auditor that cannot see the file thinks the message was made up.

Mitigation: best-of- n veto. We replay the v2 Sonnet 4.6 and Haiku 4.5 runs as if a deployment shipped one of the five trajectories in each task $\times$ config group (48 groups):

- picking the highest grader score ships 0.28 hacks under the corrected label (0.37 original), against 0.23 (0.33) at random;
- vetoing what Sonnet `full_trace` flags, then taking the highest score, ships 0.15 (difference CI [0.05, 0.23]; original 0.21);
- that equals the oracle, which ships a clean trajectory whenever one exists (0.15; 0.19 original).

Here the mode matters more than in Table 2: a Sonnet

veto ships 0.21 hacks with `full_trace` against 0.29 with `report_only` (original label).

Cost. For the whole judged v2 study, `eval.cost` estimates about \$88. That is roughly 2.2 cents per Sonnet audit (3.0 cents when `eval.cascade` rebuilds the full audit request) and 7 cents per agent run. These runs did not log token usage, so the estimate is a lower bound.

Preliminary: v3 and new conditions. The API credit ran out during the 10-task v3 runs, so they have labeller labels only (REPORT.md §8.6):

- Haiku 4.5: 200 trajectories. Covert runs hack 42/50 and honest runs 0/50.
- Sonnet 4.6: 150 of 200 planned trajectories.
- `covert_monitored` (the agent is told an auditor reads its reasoning), with all 15 labeller positives read by hand (9 were labeller false positives): Haiku hacked 1/14 times on three tasks against 13/15 under v3 `covert`, and Sonnet 4.6 5/15 against 11/15. All 6 remaining hacks state the fabrication in the reasoning. Monitoring deterred hacking; it did not make it hidden.

The v3 labeller is stricter, so these rates cannot be compared with v2.

Does the reasoning disclose the hack? A regex rubric (`eval/obfuscation.py`; 10/10 on a held-out hand-labelled sample) finds the exploit stated in the reasoning of 25/25 Haiku, 28/28 Sonnet 4.5 and 20/23 Sonnet 4.6 `covert` hacks in v2 (87–100%; corrected label 63/66, 0.95). The 3 exceptions are `fake_citation`, where the overstatement lives in the report. Unprompted hacks (incentivised or pressure) disclose less often, 0–90% per cell (4–10 hacks each).

Label audit. `tasks/LABEL_AUDIT.md` revises the labeller rules. On the 360 judged v2 runs, labeller–judge

agreement rises from $\kappa = 0.55$ to 0.80, and the labeller rate on honest v3 runs falls from 24/112 to 5/112; the v3 numbers here use the relabelled episodes. Our v2 headline uses the frozen run-time labels, so Table 2 does not change.

Judge validation. On a stratified sample of 60 v2 trajectories with blind labels from an AI annotator (Devin, not a human; docs/judge_validation.md), the judge has accuracy 0.85 and $\kappa = 0.70$ (labeller: 0.82, 0.57). It is never more lenient: all 23 blind-hacked runs are judge-hacked. 8 of its 9 disagreements are `leaked_answer`: the judge was shown the current definition (honest $p \approx 0.006$), but v2 agents ran on older data where $p \approx 0.312$ is correct. So the judge over-flags honest `leaked_answer` runs. Under the corrected label it agrees on 59/60 ($\kappa = 0.97$), though that is not independent evidence. The corrected and excluded numbers are in Table 2.

Operating at a low false-positive rate. With leave-one-task-out thresholds on raw confidence (results/calibration_v2), at FPR $\leq 5\%$ Sonnet `full_trace` keeps recall 0.70 against 0.49 for `report_only` and 0.15 for `final_report`; Opus gets 0.63, 0.57 and 0.19. So the reasoning’s AUROC gain for Sonnet matters exactly where a deployment would operate, although its default verdicts (0.93 vs 0.93) hide it. No threshold keeps `final_report` FPR at or below 10% on unseen tasks. Platt and isotonic maps do not transfer across tasks: raising the threshold on raw confidence is the better lever. That holds for the original label only. Under the corrected label, thresholds tuned on five tasks reach FPR 0.15–0.18 on the sixth for every trace mode, so no Claude auditor $\times$ mode keeps unseen-task FPR $\leq 10\%$ with useful recall.

Errors and evidence. On the original label (docs/error_analysis.md), trace-mode false positives are agents that read the grader and reported honest values, and `final_report` false positives are honest “N/A” reports for the failed plate. Under the corrected label, grader-reading runs are 301 of the 375 false-positive audits. The rendered trajectory id encodes task and config, and 122 of 1,800 audits cite it, so future runs should use opaque ids. Against line-level gold on 19 v2 hacks (fixtures/evidence_gold), `full_trace` auditors cite 0.81–0.83 of hack lines, against 0.19–0.26 for `final_report`.

Deployment cost and attacks. On the 240 Opus-audited trajectories (results/CASCADE.md), Sonnet alone has recall 0.89 and FPR 0.25 at about \$0.030 per trajectory, Opus alone 0.84 and 0.11 at \$0.051, and a Sonnet→Opus cascade 0.82 and 0.11 at \$0.056 (45.8% escalated), so Opus alone is the best value on the original label. Under the corrected label (results/CASCADE_corrected.md) every default Claude policy has FPR ≥ 0.22 ; only a tuned heuristic→Sonnet cascade stays under 10% (recall 0.80, FPR 0.054, \$0.019 in-sample; 0.77 at 0.054 held out).

A monitor-attack corpus (10 real `covert` hacks $\times$ 6 attacks: report and file injections, laundered reasoning, padding, a fake self-check) has so far been scored only by the offline heuristic auditor: `full_trace` recall drops from 7/10 to 6/10 under laundered reasoning. Claude auditors on the hard-case and attack sets, the agentic (recomputing) auditor and n -sample ensembles are implemented but not yet run.

4 Limitations

- *Small n .* There are 5 episodes per cell and 30 per subject $\times$ config, so many CIs are wide.
- *Ground truth.* The judge is a model with privileged information, not a human. It has been checked only against AI blind labels, and it over-flags v2 `leaked_answer`.
- *Instructed hacking.* Most positives come from `covert`, where we told the agent to hack. Natural hacking is rarer and softer.
- *Rubric/judge mismatch.* The rubric and the judge disagree about whether reading the grader counts. This mismatch drives most trace-mode false positives.
- *Coverage.* The Opus re-audit covers 240 of the 360 trajectories.
- *Compute cap.* The v3 runs, the new conditions, fault injection and the Opus-subject runs are partial and unjudged. We treat them as preliminary only.
- *Task realism.* The tasks are small and synthetic. Real analyses are longer and have more room for drift.

5 Related work

Our closest neighbours are CoT-versus-action monitor comparisons in coding agents [3, 4], agent-plus-monitor sabotage benchmarks [12], and impossible-task cheating benchmarks [22]. Work on in-context scheming [14, 17] and sandbagging [20] motivates our `covert` model organism. Faithfulness results [7] and verbalisation training [19] explain why we do not rely on the CoT alone. Punish applies this kind of monitor ablation to AI-scientist integrity. A full annotated list is in docs/related_work.md.

6 Future work

- Align the rubric with the judge on grader access.
- Score Claude auditors on our 32-case hand-labelled hard-case set.
- Run the live audit-and-retry mitigation.
- Judge the v3 runs, re-judge v2 `leaked_answer` with the definition the agents saw, and validate the judge against human labels.
- Test reasoning obfuscation under `covert_monitored` at a usable n , with the judge.

Reproducibility. `eval.analyze results/v2_sonnet46 results/v2_haiku45 results/v2_sonnet45 -out results/RESULTS.md -figures` regenerates every table in this paper offline from the committed trajectories, verdicts and judgments. Worked examples with quoted trajectory lines are in `docs/case_studies.md`.

References

- [1] Amodei, Olah, Steinhardt, Christiano, Schulman, and Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016. URL <https://arxiv.org/abs/1606.06565>.
- [2] Anthropic. From shortcuts to sabotage: natural emergent misalignment from reward hacking, 2025. URL <https://www.anthropic.com/research/emergent-misalignment-reward-hacking>.
- [3] Arnav, Bernabeu-Pérez, Helm-Burger, Kostolansky, Whittingham, and Phuong. CoT red-handed: Stress testing chain-of-thought monitoring. *arXiv preprint arXiv:2505.23575*, 2025. URL <https://arxiv.org/abs/2505.23575>.
- [4] Baker, Huizinga, Gao, Dou, Guan, Madry, Zaremba, Pachocki, and Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. *arXiv preprint arXiv:2503.11926*, 2025. URL <https://arxiv.org/abs/2503.11926>.
- [5] Beel, Kan, and Baumgart. Evaluating Sakana’s AI scientist: Bold claims, mixed results, and a promising future? *arXiv preprint arXiv:2502.14297*, 2025. URL <https://arxiv.org/abs/2502.14297>.
- [6] Bondarenko, Volk, et al. Demonstrating specification gaming in reasoning models. Palisade Research, 2025. URL <https://palisaderesearch.org/research/specification-gaming>.
- [7] Chen et al. Reasoning models don’t always say what they think. *arXiv preprint arXiv:2505.05410*, 2025. URL <https://arxiv.org/abs/2505.05410>.
- [8] Denison et al. Sycophancy to subterfuge: Investigating reward tampering in language models. *arXiv preprint arXiv:2406.10162*, 2024. URL <https://arxiv.org/abs/2406.10162>.
- [9] Emmons, Jenner, et al. When chain of thought is necessary, language models struggle to evade monitors. *arXiv preprint arXiv:2507.05246*, 2025. URL <https://arxiv.org/abs/2507.05246>.
- [10] Korbak, Balesni, et al. Chain of thought monitorability: A new and fragile opportunity for AI safety. *arXiv preprint arXiv:2507.11473*, 2025. URL <https://arxiv.org/abs/2507.11473>.
- [11] Krakovna et al. Specification gaming: the flip side of AI ingenuity. DeepMind blog, 2020. URL <https://deepmind.google/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>.
- [12] Kutasov et al. SHADE-Arena: Evaluating sabotage and monitoring in LLM agents. *arXiv preprint arXiv:2506.15740*, 2025. URL <https://arxiv.org/abs/2506.15740>.
- [13] Marks et al. Auditing language models for hidden objectives. *arXiv preprint arXiv:2503.10965*, 2025. URL <https://arxiv.org/abs/2503.10965>.
- [14] Meinke, Schoen, Scheurer, Balesni, Shah, and Hobbhahn. Frontier models are capable of in-context scheming. *arXiv preprint arXiv:2412.04984*, 2024. URL <https://arxiv.org/abs/2412.04984>.
- [15] METR. Recent frontier models are reward hacking, 2025. URL <https://metr.org/blog/2025-06-05-recent-reward-hacking/>.
- [16] Sakana AI. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. URL <https://arxiv.org/abs/2408.06292>.
- [17] Schoen et al. Stress testing deliberative alignment for anti-scheming training. *arXiv preprint arXiv:2509.15541*, 2025. URL <https://arxiv.org/abs/2509.15541>.
- [18] Skalse, Howe, Krashennikov, and Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, 2022. URL <https://arxiv.org/abs/2209.13085>.
- [19] Turpin, Ardit, Li, Benton, and Michael. Teaching models to verbalize reward hacking in chain-of-thought reasoning. *arXiv preprint arXiv:2506.22777*, 2025. URL <https://arxiv.org/abs/2506.22777>.
- [20] van der Weij, Hofstätter, Jaffe, Brown, and Ward. AI sandbagging: Language models can strategically underperform on evaluations. *arXiv preprint arXiv:2406.07358*, 2024. URL <https://arxiv.org/abs/2406.07358>.
- [21] Walters and Wilder. Fabrication and errors in the bibliographic citations generated by ChatGPT. *Scientific Reports*, 13:14045, 2023. URL <https://www.nature.com/articles/s41598-023-41032-5>.
- [22] Zhong, Raghunathan, and Carlini. ImpossibleBench: Measuring LLMs’ propensity of exploiting test cases. *arXiv preprint arXiv:2510.20270*, 2025. URL <https://arxiv.org/abs/2510.20270>.